

# Day 59: Vanishing & Exploding Gradients

MD Mutasim Billah | 52-Week Data Science → ML/AI Roadmap | Week 9: Sequence Data & RNNs I  
(Day 4 of 7)

---

## 0. Where Today Fits

Day 58 trained an RNN at  $T=40$  -- but only because its task let the network LATCH onto large  $\pm 3$  spikes, routing around the vanishing-gradient problem instead of exposing it. Today uses a task chosen so that can't happen, and measures both failure modes directly: gradients that shrink to nothing on the way back through time, and gradients that blow up. The last experiment is the baseline Day 60's gradient clipping will be tested against.

### How to read each section

Every experiment runs **Specification** (exact setup, sizes, what counts as success) → **Why this test** → **Implementation** (the real code from `day59_vanishing_exploding_gradients.py`) → **Actual output** (verbatim from `day59_run_output.txt`) → **What it means**. Try to predict each output before reading it.

Watch: [Stanford CS224N: NLP with Deep Learning | Winter 2019 | Lecture 7 – Vanishing Gradients, Fancy RNNs](#)

Watch: [MIT 6.S191 \(2020\): Recurrent Neural Networks](#)

## 1. The Task, and Why Gradients Vanish or Explode

### SPECIFICATION

|                     |                                                                                      |
|---------------------|--------------------------------------------------------------------------------------|
| <b>Input</b>        | $T$ scalars, EVERY one drawn from $N(0, 1)$ -- $x_0$ included                        |
| <b>Label</b>        | 1 if the FIRST input $x_0 > 0$ , else 0. The other $T-1$ inputs are pure distraction |
| <b>Chance level</b> | accuracy 0.5 (classes are balanced in expectation)                                   |
| <b>Model</b>        | Day 58's VanillaRNNClassifier, unchanged: hidden 16, tanh, many-to-one, plain SGD    |

### Why this task

$x_0$  is no bigger than any input after it, so a 'latch onto large inputs' strategy -- what rescued Day 58 -- would latch onto the wrong input. The only way to solve it is for gradient to travel from the final timestep all the way back to step 0. That makes it a clean test of exactly the thing that can fail.

## MATH

From Day 58's BPTT, one step back in time is:

$$dL/dh_{t-1} = (dL/dh_t * (1 - h_t^2)) @ Whh^T$$

So  $k$  steps back is a **PRODUCT** of  $k$  factors, each involving  $Whh$  and  $\tanh$ 's slope  $(1 - h_t^2)$ , which is at most 1. Roughly, the gradient scales like  $(\text{per-step gain})^k$ : below 1 it shrinks exponentially (vanishing), above 1 it grows exponentially (exploding). The per-step gain depends on  $Whh$ 's **spectral radius**  $\rho$  -- its largest  $|\text{eigenvalue}|$  -- **TIMES** how steep  $\tanh$  is at the current state.

## IMPLEMENTATION

```
def make_sign_task(n, T, seed):
    rng = np.random.RandomState(seed)
    X = rng.randn(n, T)
    y = (X[:, 0] > 0).astype(int)
    return X, y

def spectral_radius(W):
    return np.max(np.abs(np.linalg.eigvals(W)))

def set_spectral_radius(net, rho):
    net.Whh *= rho / spectral_radius(net.Whh) # scaling W scales every eigenvalue equally
```

## 2. A New Probe -- and Verifying It First

### SPECIFICATION

|                       |                                                                                                                                                                                          |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Probe</b>          | <code>grad_by_timestep(net, X, y)</code> : Day 58's BPTT loop, but it <b>RECORDS</b> $dL/dh_t$ at every $t$ (true gradient of the mean loss, so <code>dlogits</code> is divided by $m$ ) |
| <b>Problem</b>        | a hidden state $h_t$ can't be nudged directly for a numeric check                                                                                                                        |
| <b>Solution</b>       | inputs <b>CAN</b> be nudged. The probe implies $dL/dx_t = (dL/dh_t * (1 - h_t^2)) @ Wxh^T$ . If that matches numeric differences for every $x_t$ , every recorded $dL/dh_t$ is right     |
| <b>Check setup</b>    | <code>VanillaRNNClassifier(hidden_dim=4, seed=1)</code> ; 3 sign-task sequences of length 8 (seed 1)                                                                                     |
| <b>Checked</b>        | all 24 inputs; central difference, $\text{eps} = 1e-5$                                                                                                                                   |
| <b>Pass criterion</b> | max relative error $< 1e-5$                                                                                                                                                              |

### Why this test

Every number in Sections 3 and 4 comes from this probe. A probe with an off-by-one timestep or a missing  $\tanh$  factor would produce plausible-looking curves that are simply wrong. Checking it through the inputs verifies the entire recorded sequence, not just one value.

## IMPLEMENTATION

```
def grad_by_timestep(net, X, y):
    net.forward(X)
    m, T = X.shape
    dlogits = (net.probs - one_hot(y, 2)) / m    # TRUE gradient of the mean loss
    dh = dlogits @ net.Why.T                    # dL/dh_{T-1}
    dH = np.zeros_like(net.H)
    for t in reversed(range(T)):
        dh[:, t, :] = dh                        # RECORD -- the only addition to Day 58's loop
        da = dh * (1.0 - net.H[:, t, :] ** 2)    # undo tanh
        dh = da @ net.Whh.T                      # back to h_{t-1}
    return dh

# the check: the dL/dx_t the probe implies...
for t in range(T):
    da = dH[:, t, :] * (1.0 - net.H[:, t, :] ** 2)
    dX_analytic[:, t] = (da @ net.Wxh.T)[: , 0]
# ...versus nudging each input X[i, t] by +/- eps and watching the loss
```

## ACTUAL OUTPUT

```
checked 24 input entries (3 sequences x 8 timesteps)
sample, example 0: t=0 analytic -8.473e-05 | t=7 analytic -7.495e-01
max relative error: 1.96e-08 (PASS -- the probe reports true gradients)
```

### What it means

Agreement to about eight significant figures on all 24 inputs. Note the sample: the gradient at  $t=0$  is about 10,000x smaller than at  $t=7$  -- vanishing is already visible in an 8-step sequence.

## 3. Gradient vs. Distance -- Sweeping the Spectral Radius

### SPECIFICATION

|                     |                                                                                                                                     |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <b>Networks</b>     | untrained VanillaRNNClassifier(hidden_dim=16, seed=3), Whh rescaled to $\rho = 0.5, 0.9, 1.0, 1.5, 2.0, 3.0$ -- otherwise identical |
| <b>Data</b>         | 200 sign-task sequences, $T = 50$ , inputs $N(0, 1)$ (seed 0), shared by every $\rho$                                               |
| <b>Measured</b>     | $\ dL/dh_t\ $ (Frobenius norm over the batch) at $t = 49, 40, 25, 0$ , and the ratio $dh_0 / dh_{49}$                               |
| <b>Second sweep</b> | the SAME networks and sequences with every input scaled by 0.1, $\rho = 0.9, 1.0, 1.2, 1.5, 2.0$                                    |

### Why this test

It isolates one variable -- Whh's spectral radius -- and measures its effect on how much gradient survives 49 steps back. The textbook rule says  $\rho < 1$  vanishes and  $\rho > 1$  explodes. The second sweep tests whether that rule depends on tanh's slope, by shrinking the inputs so tanh stays nearly linear.

## IMPLEMENTATION

```
X, y = make_sign_task(200, T=50, seed=0)
for rho in [0.5, 0.9, 1.0, 1.5, 2.0, 3.0]:
    net = VanillaRNNClassifier(hidden_dim=16, seed=3) # same seed -> same weights up to the rescale
    set_spectral_radius(net, rho) # the ONLY difference
    n = grad_norms(grad_by_timestep(net, X, y)) # ||dL/dh_t|| for every t
    print(rho, n[49], n[40], n[25], n[0], n[0] / n[49])

for rho in [0.9, 1.0, 1.2, 1.5, 2.0]: # second sweep: small inputs
    ...
    n = grad_norms(grad_by_timestep(net, X * 0.1, y))
```

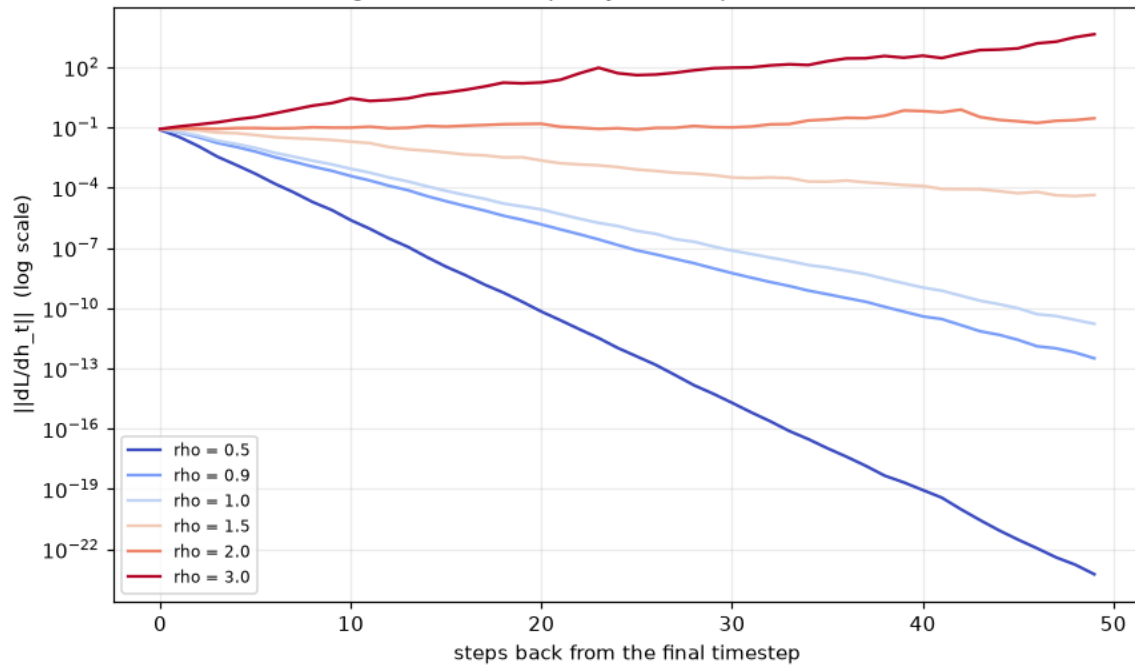
## ACTUAL OUTPUT

| rho | dh_49     | dh_40     | dh_25     | dh_0      | dh_0/dh_49 |
|-----|-----------|-----------|-----------|-----------|------------|
| 0.5 | 7.656e-02 | 7.815e-06 | 1.058e-12 | 5.690e-24 | 7.43e-23   |
| 0.9 | 8.180e-02 | 6.992e-04 | 1.413e-07 | 3.222e-13 | 3.94e-12   |
| 1.0 | 8.276e-02 | 1.462e-03 | 1.244e-06 | 1.674e-11 | 2.02e-10   |
| 1.5 | 8.568e-02 | 2.400e-02 | 1.072e-03 | 4.333e-05 | 5.06e-04   |
| 2.0 | 8.478e-02 | 9.862e-02 | 9.191e-02 | 2.874e-01 | 3.39e+00   |
| 3.0 | 8.483e-02 | 1.647e+00 | 5.082e+01 | 4.410e+03 | 5.20e+04   |

## ACTUAL OUTPUT

| rho | dh_0/dh_49 |
|-----|------------|
| 0.9 | 2.74e-04   |
| 1.0 | 2.11e-02   |
| 1.2 | 2.92e+00   |
| 1.5 | 4.06e+01   |
| 2.0 | 1.09e+03   |

Gradient reaching earlier timesteps, by Whh's spectral radius (untrained, T=50)



### What it means

Both regimes appear in one sweep. At  $\rho=0.5$  the gradient reaching step 0 is  $\sim 1e-22$  of what arrives at the end -- effectively zero. At  $\rho=3.0$  it is  $\sim 50,000\times$  LARGER -- exploding. And the boundary is not the textbook  $\rho=1$ : with  $N(0,1)$  inputs even  $\rho=1.0$  vanishes ( $2e-10$ ), and gradient is roughly preserved near  $\rho=2$ . The second sweep explains why: with inputs shrunk so  $\tanh$  stays in its linear region (slope near 1), the boundary moves back to about 1.2. The per-step gain is  $\rho$  TIMES  $\tanh$ 's slope, and  $N(0,1)$  inputs push units partway into saturation, where the slope is well below 1.

## 4. The Real Cost -- Trainability vs. Sequence Length

### SPECIFICATION

|                             |                                                                                                                                                                                                |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Lengths</b>              | $T = 2, 5, 10, 20, 40$                                                                                                                                                                         |
| <b>Data (per length)</b>    | train 600 sign-task sequences (seed 42), test 400 (seed 999)                                                                                                                                   |
| <b>Network</b>              | VanillaRNNClassifier(hidden_dim=16, lr=0.05, seed=7) -- default init, identical every time                                                                                                     |
| <b>Training</b>             | 150 epochs, mini-batch 30, plain SGD -- identical every time                                                                                                                                   |
| <b>Measured</b>             | train/test accuracy; gradient ratio $\ dL/dh_0\  / \ dL/dh_{\{T-1\}}\ $ on the test set, at initialization AND after training                                                                  |
| <b>Follow-up (T=20, 40)</b> | how far the trained final hidden state moves when (a) $x_0$ 's sign is flipped vs. (b) one mid-sequence input is resampled; and the best  correlation  of any final hidden unit with the label |

### Why this test

Section 3 was untrained networks. This asks whether the effect actually stops LEARNING. The gradient ratio at initialization matters most: training starts there, and if almost no gradient reaches  $x_0$ , the network never gets the signal to start using it. The follow-up checks directly whether the failing networks stored  $x_0$ : flipping  $x_0$  should move the final state a lot (it flips the answer); resampling a noise input shouldn't.

### IMPLEMENTATION

```
for T in [2, 5, 10, 20, 40]:
    X_tr, y_tr = make_sign_task(600, T, seed=42)
    X_te, y_te = make_sign_task(400, T, seed=999)
    net = VanillaRNNClassifier(hidden_dim=16, lr=0.05, seed=7)
    n_init = grad_norms(grad_by_timestep(net, X_te, y_te)) # BEFORE training
    train(net, X_tr, y_tr, epochs=150, batch_size=30, seed=7)
    n_end = grad_norms(grad_by_timestep(net, X_te, y_te)) # AFTER training
    print(T, train_acc, test_acc, n_init[0] / n_init[-1], n_end[0] / n_end[-1])

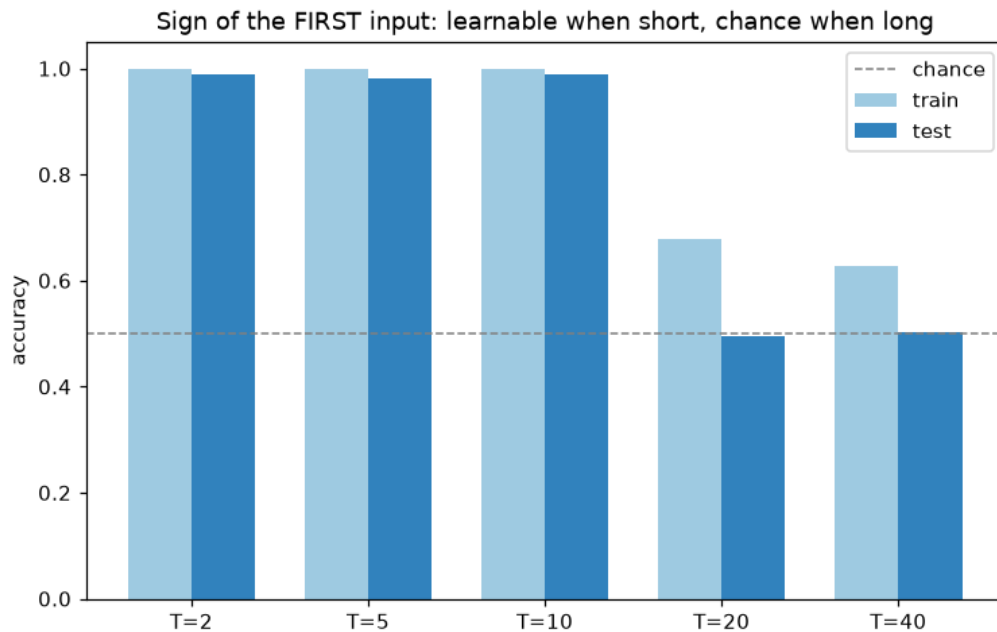
# follow-up, for T = 20 and 40
X_flip = X_te.copy(); X_flip[:, 0] *= -1 # flip the input that matters
X_mid = X_te.copy(); X_mid[:, T // 2] = rng.randn(len(y_te)) # resample one that doesn't
move_flip = norm(h_T(X_flip) - h_T(X_te), axis=1).mean()
move_mid = norm(h_T(X_mid) - h_T(X_te), axis=1).mean()
```

### ACTUAL OUTPUT

| T  | train_acc | test_acc | ratio@init | ratio@end |
|----|-----------|----------|------------|-----------|
| 2  | 1.000     | 0.990    | 9.07e-01   | 3.00e+00  |
| 5  | 1.000     | 0.980    | 5.39e-01   | 9.87e+00  |
| 10 | 1.000     | 0.990    | 2.08e-01   | 3.06e+01  |
| 20 | 0.678     | 0.495    | 1.79e-02   | 7.80e-03  |
| 40 | 0.627     | 0.502    | 3.29e-04   | 3.97e-01  |

### ACTUAL OUTPUT

| T  | move from flipping $x_0$ | move from resampling $x_{mid}$ | max  corr  with label |
|----|--------------------------|--------------------------------|-----------------------|
| 20 | 0.010                    | 0.140                          | 0.070                 |
| 40 | 0.057                    | 0.308                          | 0.107                 |



### What it means

A clean threshold. The gradient ratio at initialization falls steadily -- 0.91, 0.54, 0.21, 0.018, 0.00033 -- and learning succeeds down to 0.21 (T=10, test 0.990) and fails from 0.018 on (T=20 and 40, test at chance). The failing networks never stored  $x_0$ : flipping it moves the final state 5-14x LESS than resampling one irrelevant noise input, and no hidden unit correlates with the label above 0.11. Train accuracy above chance (0.678, 0.627) with test at chance is memorization of noise, not learning of the rule.

### WHY THIS CHOICE

A correction worth remembering: the gradient ratio AFTER training at T=40 is 0.40 -- not small at all. Read alone, that would suggest the gradient didn't vanish. But by the end of training the failing networks' Whh has drifted into the large-spectral-radius regime (Section 5), where gradient flows back without carrying a usable learning signal. The measurement that explains the failure is the one at INITIALIZATION -- where learning has to start -- and the direct check of what the trained network actually stored.

## 5. Exploding Gradients During Ordinary Training

### SPECIFICATION

|                           |                                                                                                                                                    |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Run</b>                | Section 4's T=20 run exactly (same data, network, default init, lr, epochs, batch size) -- re-run while recording                                  |
| <b>Recorded each step</b> | global gradient norm = sqrt(sum of squares of all five gradients); batch loss before the update; Whh's spectral radius after it. 3,000 steps total |
| <b>Spike</b>              | any step whose gradient norm exceeds 10x the run's median                                                                                          |
| <b>Comparison</b>         | the same training started from Whh rescaled to $\rho = 3.0$ and $4.0$                                                                              |

### Why this test

Exploding is easy to show with a deliberately bad initialization. The more important question is whether it happens with the DEFAULT one. Section 3 found gradients start exploding above  $\rho$  of about 2 -- so tracking Whh's radius alongside the gradient norm tests whether training itself pushes the network across that line.

### IMPLEMENTATION

```
def train_tracked(net, X, y, epochs, batch_size, seed):
    ...
    loss.append(cross_entropy_loss(net.forward(X[idx]), y_oh))    # before the update
    net.backward(y_oh)                                           # grads + update
    gnorm.append(np.sqrt(sum(np.sum(g ** 2) for g in net.grads.values()))))
    radius.append(spectral_radius(net.Whh))                      # after the update

g, L, R = train_tracked(net, X_tr, y_tr, epochs=150, batch_size=30, seed=7)
med = np.median(g)
spikes = np.where(g > 10 * med)[0]
cross = np.argmax(R > 2.0)                                     # first step past the boundary
dL = np.abs(np.diff(L))                                         # step-to-step loss change
after = dL[spikes]                                              # the change each spike's update caused
```

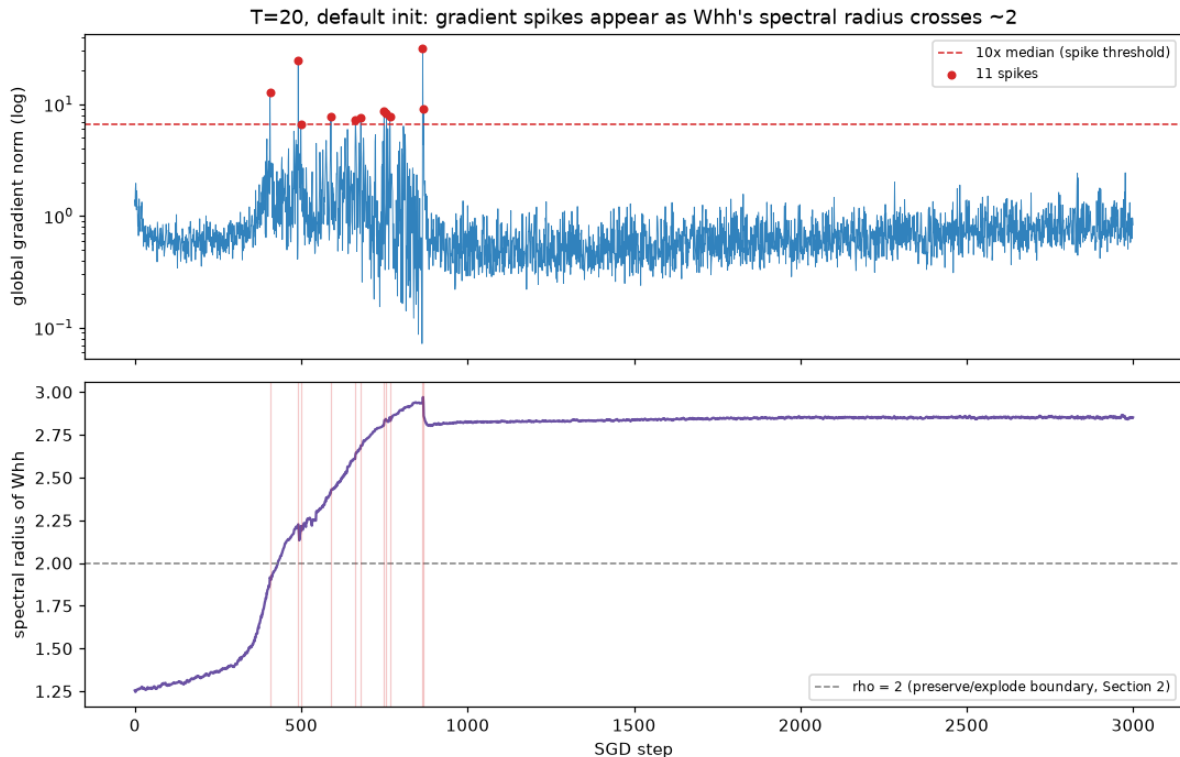
### ACTUAL OUTPUT

```
Whh spectral radius: 1.25 at init -> 2.20 at step 500 -> 2.85 at the end
first step with radius > 2.0: 430
gradient norm: median 0.663, max 31.4 (47x the median)
spikes (> 10x median): 11, at steps [406, 491, 499, 589, 663, 679, 749, 756, 766, 865,
868]
spikes while radius <= 2.0: 1   while radius > 2.0: 10
|batch-loss change| per step: median 0.042, right after a spike 0.563
```

### ACTUAL OUTPUT

| init rho | median gnrm | max gnrm | spikes | final rho |
|----------|-------------|----------|--------|-----------|
| 1.25*    | 0.663       | 31.4     | 11     | 2.85      |
| 3.00     | 0.416       | 148.1    | 40     | 7.33      |
| 4.00     | 0.476       | 1909.2   | 52     | 16.28     |

(\* = default initialization)



### What it means

Exploding gradients happen with the default initialization. Training pushes Whh's spectral radius from 1.25 up to 2.85, crossing 2.0 at step 430. The 11 spikes all fall between steps 406 and 868 -- ten of them after the crossing -- reaching 47x the median gradient norm. Each spike's update jolts the batch loss by 0.563 on average, 13x a typical step's 0.042. Then the spikes stop: once the radius settles high, units saturate and gradients shrink again. Starting from a larger radius makes it far worse -- at  $\rho=4$ , gradient norms reach 1,909, about 4,000x the median.

## 6. What Day 60 Tests

At T=20 two different problems occur together: gradient reaching early timesteps VANISHES, so the dependency on  $x_0$  is never learned; and the global gradient norm occasionally EXPLODES, jolting the weights. Gradient clipping -- capping the gradient's norm before each update -- is the standard fix for the second. Day 60 applies it to exactly Section 5's run and reports, honestly, what it fixes and what it can't.

## Interview Preparation -- Technical Questions

### Q: Why do RNN gradients vanish or explode?

A: Backpropagating  $k$  steps multiplies  $k$  factors of  $(\tanh \text{ slope}) \times \text{Whh}^T$ . A product of  $k$  similar factors behaves like  $(\text{per-step gain})^k$  -- exponential decay below 1, exponential growth above 1.

### Q: Why isn't the boundary between vanishing and exploding simply spectral radius 1?

A: Because each step also multiplies by  $\tanh$ 's slope  $(1 - h^2)$ , which is at most 1. Measured here: with  $N(0,1)$  inputs gradient is roughly preserved near  $\rho=2$ ; with inputs shrunk so  $\tanh$  stays linear, near 1.2.

**Q: How can you verify a gradient for a hidden state you can't perturb directly?**

A: Check a quantity it implies that you CAN perturb. Here  $dL/dh_t$  implies  $dL/dx_t$  through the tanh step and  $Wxh$ ; matching that against numeric input nudges verifies every recorded  $dL/dh_t$ .

**Q: A trained network's final gradient ratio at T=40 was 0.40, yet it failed. Why isn't that a contradiction?**

A: The ratio was measured after training had pushed  $Whh$  into a large-radius regime. What matters is the gradient at initialization, when learning must begin -- there it was  $3.3e-4$  -- and whether the network stored  $x_0$ , which a direct check showed it did not.

## Interview Preparation -- Theory Questions

**Q: Why did Day 58's task train at T=40 while today's fails at T=20?**

A: Day 58's signal (+3 spikes over small noise) was large enough to drive units into saturation and latch -- a nonlinear memory the network can learn from short-range examples. Today's  $x_0$  is the same size as every distractor, so nothing latches; learning depends on gradient reaching step 0, and it doesn't.

**Q: Why do gradient spikes appear mid-training rather than at the start?**

A: Training grows  $Whh$ 's spectral radius. Spikes appear as it crosses the boundary where per-step gain exceeds 1 -- the regime Pascanu et al. (2013) described as gradient 'cliffs'.

**Q: Can gradient clipping fix vanishing gradients?**

A: No. Clipping only SHRINKS gradients that are too large; it cannot enlarge ones that are too small. That is why Day 60 treats the two problems separately.

## Key Takeaway and What's Next

**Key takeaway:** backpropagating through time multiplies one factor per step, so gradient reaching early timesteps shrinks or grows exponentially. Measured directly:  $\sim 1e-22$  of the gradient survives 49 steps at  $\rho=0.5$ ,  $\sim 50,000\times$  MORE at  $\rho=3.0$ , with the boundary near  $\rho=2$  because of tanh's slope. On a task nothing can latch onto, an RNN learns at  $T \leq 10$  and fails at chance from  $T=20$ . And with default settings, training itself drives  $Whh$  across the boundary, producing gradient spikes of 47x the median.

**What's next:** Day 60 builds gradient clipping from scratch and tests it on Section 5's run.

Suggested watch order: 1) Stanford CS224N Lecture 7 -- Vanishing Gradients, Fancy RNNs, 2) MIT 6.S191 (2020): Recurrent Neural Networks.